SOLID Case Study Submission

Name: P. Latheesh Reddy
Registration Number: 23BCE0329
Course: BCSE102L - Structured and Object-Oriented Programming
Project: Campus Esports Tournament Registration System

Part A - Understanding SOLID

1. Single Responsibility Principle (SRP)

Explanation: A class should have only one reason to change, meaning it should only handle one specific job. As software grows, SRP keeps bugs isolated; changing how we save data won't accidentally break how a player's profile is displayed.

C++ Example & Violation: A violation would be a `Player` class that holds player stats (like game ID) *and* has a `saveToFile()` function. If the file format changes from text to JSON, the `Player` class has to change.
Correction: Create a `Player` class just for data, and a separate `PlayerDatabase` class responsible for saving/loading.

2. Open/Closed Principle (OCP)

Explanation: Software entities should be open for extension but closed for modification. We should be able to add new features without altering existing, tested code. This makes scaling easy because new requirements mean writing new classes rather than modifying old logic.

C++ Example & Violation: A violation is having a `TournamentRegistration` class with a `calculateFee()` function full of `if-else` statements for "Solo", "Duo", and "Squad".
Correction: Create a base class `RegistrationType` with a virtual `getFee()` method. Create subclasses like `SoloRegistration`. To add a new mode, we just write a new subclass.

3. Liskov Substitution Principle (LSP)

Explanation: Objects of a superclass should be replaceable with objects of its subclasses without breaking the application. A derived class must honor the contract established by the base class.

C++ Example & Violation: Suppose we have a base class `Match` with a `startServer()` method. If we create a subclass `OfflineLANMatch`, it might throw an exception for `startServer()` because it doesn't need an online server. This violates LSP.
Correction: Separate the hierarchy. Create a base `Match` class, and two separate interfaces for `OnlineMatch` and `OfflineMatch`.

4. Interface Segregation Principle (ISP)

Explanation: Clients should not be forced to depend on interfaces they do not use. Instead of one giant interface, we should have multiple smaller, specific ones.

C++ Example & Violation: A violation is a massive `IUser` interface with `playMatch()`, `register()`, `banPlayer()`. A standard student is forced to implement `banPlayer()`. *Correction:* Split it into `IPlayer` (with `playMatch()`, `register()`) and `IAdmin` (with `banPlayer()`).

5. Dependency Inversion Principle (DIP)

Explanation: High-level modules should not depend on low-level modules; both should depend on abstractions. This makes it easy to swap out low-level implementations without rewriting core business logic.

C++ Example & Violation: A violation is a `TournamentManager` class directly instantiating an `EmailNotifier` class. If we switch to SMS notifications, we have to rewrite `TournamentManager`. *Correction:* Create an `INotificationService` interface. Have `EmailNotifier` implement it, and pass `INotificationService` into the constructor of `TournamentManager`.

Part B - Project Implementation

GitHub Repository Link: [Insert your public GitHub repo link here]
Contributors: P. Latheesh Reddy (23BCE0329) - Sole contributor

README Outline:
Problem: The campus needs an event registration system for esports tournaments that can scale as new games, team sizes, and notification methods are added.
Features: Registering players/teams, calculating fees based on match type, and sending confirmation notifications.
Demonstrated Extension: Added an SMSNotification service alongside the standard EmailNotification to show how the design accommodates a new requirement without modifying core code.

Part 1: Interfaces and Basic Implementations

```
#include <iostream>
#include <string>
#include <vector>
#include <memory>

// 1. ISP: Small, segregated interfaces
class INotificationService {
public:
    virtual void send(const std::string& message) = 0;
    virtual ~INotificationService() = default;
};

class IFeeCalculator {
public:
    virtual double calculateFee() const = 0;
    virtual ~IFeeCalculator() = default;
};

// 2. SRP: This class ONLY handles email formatting/sending
class EmailNotification : public INotificationService {
public:
    void send(const std::string& message) override {
        std::cout << "Sending Email: " << message << "\n";
    }
};

// Extension (Part B requirement): Adding SMS doesn't change existing code (OCP)
class SMSNotification : public INotificationService {
public:
    void send(const std::string& message) override {
        std::cout << "Sending SMS text: " << message << "\n";
    }
};
```

Part 2: OCP, LSP, and DIP Implementation

```
// 3. OCP & LSP: Subclasses can replace IFeeCalculator safely
class SoloRegistration : public IFeeCalculator {
public:
    double calculateFee() const override { return 150.0; }
};

class SquadRegistration : public IFeeCalculator {
public:
    double calculateFee() const override { return 500.0; }
};

// 4. DIP: Manager depends on abstractions
class RegistrationManager {
private:
    std::shared_ptr<INotificationService> notifier;
public:
    RegistrationManager(std::shared_ptr<INotificationService> n) : notifier(n) {}

    void registerTeam(const std::string& teamName, const IFeeCalculator& feeType) {
        double fee = feeType.calculateFee();
        std::cout << "Registered " << teamName << ". Amount due: Rs." << fee << "\n";
        notifier->send("Registration successful for " + teamName);
    }
};
```

Part 3: Main Function Execution

```
int main() {
    auto emailService = std::make_shared<EmailNotification>();
    auto smsService = std::make_shared<SMSNotification>();

    RegistrationManager manager(emailService);
    SoloRegistration solo;
    SquadRegistration squad;

    std::cout << "--- Campus Esports Registration ---\n";
    manager.registerTeam("Latheesh_Solo", solo);

    RegistrationManager managerSMS(smsService);
    managerSMS.registerTeam("VIT_Strikers", squad);

    return 0;
}
```

Part C - Application of SOLID

Project Overview & Class Diagram Explanation:
The project is a Campus Esports Event Registration System. The core class is `RegistrationManager`, which handles the business logic of signing up teams. It relies on two main interfaces: `INotificationService` and `IFeeCalculator`. Concrete classes like `EmailNotification`, `SMSNotification`, `SoloRegistration`, and `SquadRegistration` implement these interfaces.

How SOLID is Applied in the Code:

- **SRP:** Demonstrated by `EmailNotification`. Its only job is to handle email protocols. It does not calculate fees or manage team data.
- **OCP:** Demonstrated by the `IFeeCalculator` interface. When I needed to support a "Squad" registration alongside "Solo", I created a new class `SquadRegistration` that implements `calculateFee()`, rather than modifying the manager.
- **LSP:** Both `EmailNotification` and `SMSNotification` perfectly substitute `INotificationService`. The `RegistrationManager` calls `send()` on either one, and the application behaves correctly.
- **ISP:** Instead of a giant `ITournament` interface containing `calculateFee()` and `sendNotification()`, I segregated them. This way, the fee classes aren't forced to implement notification methods they don't need.
- **DIP:** `RegistrationManager` takes a `std::shared_ptr<INotificationService>` in its constructor. It depends on the abstraction, allowing me to inject any notification type at runtime without modifying the manager class.

Explanation of the Extension (Part B):
I demonstrated extension by adding a new notification method: SMS. Because the system was designed using DIP and OCP, all I had to do was create a new class `SMSNotification` that implements `INotificationService`. I did not have to alter a single line of code inside `RegistrationManager`.

Personal Contribution, Challenges, and Improvements:
I completed this project independently. The main design challenge was figuring out how to handle different registration types (solo, duo, squad) without cluttering the main logic. Abstracting the fee calculation behind an interface solved this. One limitation I identified is that the current `IFeeCalculator` interface is stateless. An improvement would be to allow the calculator to take in player data or a game ID (e.g., BGMI vs. Valorant) to apply dynamic pricing or campus club discounts, which would require slightly refactoring the interface signature.